

Universidad Nacional de Asunción

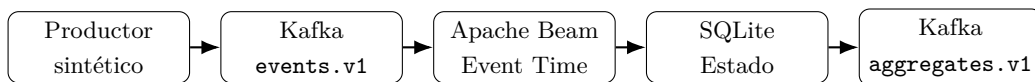
Facultad Politécnica

Maestría en Inteligencia Artificial y Análisis de Datos

Informe Técnico

Pipeline de procesamiento de eventos de pagos

Apache Kafka + Apache Beam



Autores: Diego Olmedo - Nelson Castelví

Módulo: Streaming de Datos y sus Aplicaciones

Tutor: MSc. Rodrigo Parra Zacarías

Proyecto: Proyecto Integrador de Streaming

Repositorio: [GitHub](#)

Septiembre de 2026

Índice

1. Resumen	3
2. Problema y alcance	3
2.1. Problema	3
2.2. Usuarios del resultado	4
2.3. Métricas generadas	4
3. Arquitectura de la solución	4
3.1. Diagrama de arquitectura	5
3.2. Productor sintético	5
4. Contrato de eventos	5
4.1. Descripción de los campos	6
4.2. Evolución del esquema	6
5. Apache Kafka	6
5.1. Tópicos	6
5.2. Clave de particionamiento	6
5.3. Orden de los eventos	7
6. Procesamiento con Apache Beam	7
6.1. Adaptación de Kafka a micro-batches	7
6.2. Asignación de Event Time	7
6.3. Ventanas	8
6.4. Eventos tardíos	8
7. Agregación y deduplicación	8
7.1. Agregación	8
7.2. Deduplicación	8
8. Estado persistente e idempotencia	9
8.1. Clave de idempotencia	10
9. Esquema de salida	10
10. Semántica de entrega	10
11. Pruebas y validación	11
11.1. Pruebas unitarias	11
11.2. Validación de Event Time y ventanas	11
11.3. Escenario de duplicados	11
11.4. Escenario de eventos tardíos	12
11.5. Prueba end-to-end	12

12. Operación y reproducibilidad	12
12.1. Inicio de la infraestructura	12
12.2. Instalación del entorno Python	13
12.3. Ejecución del productor	13
12.4. Ejecución del pipeline	13
12.5. Detención	13
13. Logs y observabilidad	13
14. Supuestos	14
15. Límites conocidos	14
15.1. Entrada a Beam	14
15.2. Validación formal del contrato	15
15.3. Persistencia local	15
15.4. Exactly-once	15
15.5. Retención del estado	15
16. Posibles mejoras	15
17. Conclusiones	16

1. Resumen

El presente proyecto implementa un pipeline de procesamiento de datos en streaming orientado al procesamiento de eventos de pagos. El sistema utiliza un productor sintético para generar eventos continuamente, Apache Kafka como plataforma de transporte y almacenamiento temporal de eventos, Apache Beam como motor de procesamiento y un tópico Kafka de salida para publicar las métricas agregadas.

El caso de uso fue diseñado para representar un escenario de monitoreo operativo de transacciones comerciales. Cada evento contiene un identificador único, una clave de negocio asociada al comercio, el momento en que ocurrió el evento y un conjunto de atributos relacionados con el pago.

El pipeline utiliza tiempo de evento (*Event Time*), ventanas fijas de 60 segundos y una política de *allowed lateness* de 120 segundos. Además, se implementa deduplicación mediante `event_id` y un mecanismo de estado persistente basado en SQLite que permite mantener la información acumulada entre los diferentes micro-batches procesados por Apache Beam.

La salida utiliza una clave determinística construida a partir del comercio y la ventana temporal, permitiendo una estrategia de actualización o *upsert* lógico sobre los resultados. El tópico de salida se configura con política de compactación para favorecer la conservación del estado más reciente de cada clave.

La solución fue diseñada para ejecutarse localmente mediante Docker Compose y un entorno Python administrado mediante `uv`, permitiendo reproducir la ejecución sin requerir un despliegue de producción.

2. Problema y alcance

2.1. Problema

En un sistema de pagos pueden generarse eventos continuamente desde múltiples comercios. Para fines de monitoreo operativo resulta útil obtener métricas agregadas en intervalos temporales pequeños, por ejemplo:

- cantidad total de eventos recibidos;
- cantidad de pagos confirmados;
- monto total de pagos confirmados;
- comportamiento de cada comercio dentro de una ventana temporal;
- detección y tolerancia de eventos duplicados;
- procesamiento de eventos que llegan fuera de orden o con retraso.

El desafío consiste en producir estas métricas sin depender únicamente del momento en que el sistema recibe cada evento, debido a que el orden de llegada no necesariamente coincide con el momento en que ocurrió el hecho.

2.2. Usuarios del resultado

Los resultados producidos por el pipeline pueden ser consumidos por aplicaciones o procesos destinados al monitoreo operativo de transacciones. Entre los posibles usuarios se encuentran:

- analistas de datos;
- equipos de monitoreo operativo;
- responsables de plataformas de pagos;
- sistemas de tableros o visualización;
- procesos posteriores de análisis de comportamiento comercial.

2.3. Métricas generadas

Para cada comercio y ventana temporal se calculan:

1. `total_events`: cantidad total de eventos recibidos;
2. `confirmed_events`: cantidad de eventos cuyo estado es `CONFIRMED`;
3. `total_amount`: suma de los montos correspondientes a eventos confirmados.

Estas métricas permiten construir una vista agregada del comportamiento de los pagos durante cada ventana temporal.

3. Arquitectura de la solución

La arquitectura general está compuesta por cinco bloques principales:

1. productor sintético;
2. Apache Kafka como sistema de mensajería;
3. consumidor Kafka y adaptación a micro-batches;
4. Apache Beam para procesamiento temporal y agregación;
5. SQLite y Kafka como mecanismos de persistencia y publicación de resultados.

3.1. Diagrama de arquitectura

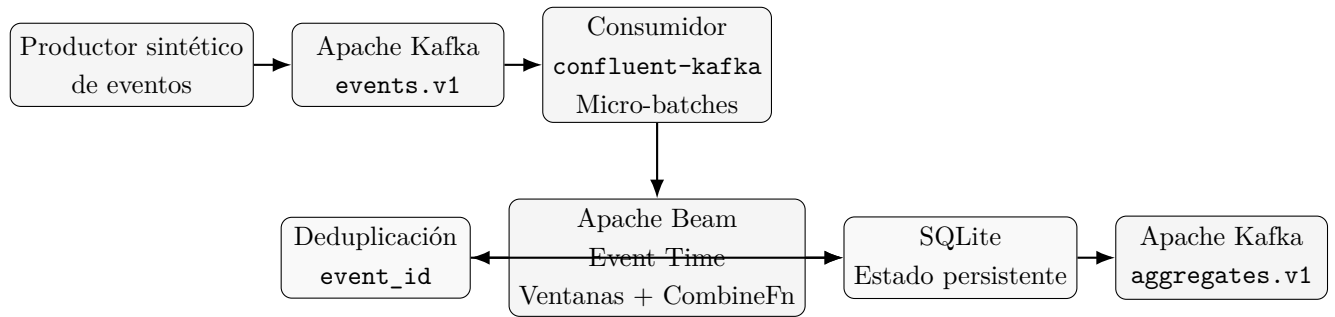


Figura 1: Arquitectura general del pipeline de streaming.

3.2. Productor sintético

El productor genera eventos de pagos continuamente. Cada evento contiene información del comercio, monto, estado y moneda.

El productor también permite simular situaciones relevantes para el procesamiento de streaming:

- eventos duplicados;
- eventos tardíos;
- generación continua de eventos;
- diferentes comercios como claves de negocio.

La simulación de duplicados utiliza nuevamente el mismo `event_id`, permitiendo evaluar el comportamiento de la estrategia de deduplicación. Los eventos tardíos se generan modificando artificialmente el `event_time` respecto del momento actual.

4. Contrato de eventos

El contrato utilizado por el productor tiene la siguiente estructura:

```

{
  "event_id": "uuid",
  "event_time": "2026-09-29T18:00:00Z",
  "merchant_id": "merchant_alpha",
  "key": "merchant_alpha",
  "payload": {
    "amount": 150.50,
    "status": "CONFIRMED",
    "currency": "USD"
  }
}

```

4.1. Descripción de los campos

Campo	Descripción
<code>event_id</code>	Identificador único del evento. Se utiliza como identificador para deduplicación.
<code>key</code>	Clave utilizada para identificar el comercio y determinar la partición de Kafka.
<code>merchant_id</code>	Identificador del comercio asociado al pago.
<code>event_time</code>	Momento en que ocurrió el evento en el dominio del negocio. Se utiliza como Event Time dentro de Apache Beam.
<code>payload.amount</code>	Monto monetario de la transacción.
<code>payload.status</code>	Estado del pago: <code>CONFIRMED</code> , <code>PENDING</code> o <code>REJECTED</code> .
<code>payload.currency</code>	Moneda de la transacción.

4.2. Evolución del esquema

La versión actual del contrato se encuentra asociada a los tópicos versionados `events.v1` y `aggregates.v1`. Esta estrategia permite distinguir una evolución futura del contrato mediante una nueva versión de los tópicos.

Como mejora futura se plantea incorporar explícitamente un campo `schema_version` dentro del evento, permitiendo gestionar diferentes versiones del contrato sin depender únicamente del versionado del tópico.

5. Apache Kafka

5.1. Tópicos

La solución utiliza dos tópicos principales:

Tópico	Función	Particiones
<code>events.v1</code>	Eventos de entrada	2
<code>aggregates.v1</code>	Resultados agregados	2

Cuadro 2: Tópicos principales del sistema.

El tópico de salida se configura con:

```
cleanup.policy=compact
```

Esto permite que Kafka pueda conservar preferentemente la versión más reciente de los registros asociados a una misma clave.

5.2. Clave de particionamiento

La clave de los eventos es:

<code>merchant_id</code>

La elección permite que los eventos correspondientes a un mismo comercio compartan la misma clave de particionamiento. Esto favorece la agrupación posterior de los eventos por comercio.

Se utilizaron dos particiones como configuración suficiente para demostrar el procesamiento distribuido a nivel de Kafka en un entorno local, manteniendo una infraestructura sencilla y adecuada al alcance académico del proyecto.

5.3. Orden de los eventos

Kafka garantiza el orden de los mensajes dentro de una partición, pero no garantiza un orden global entre todas las particiones.

Por este motivo, el pipeline no depende exclusivamente del orden de llegada. El procesamiento utiliza el campo `event_time`, permitiendo distinguir el momento del evento del momento de procesamiento.

6. Procesamiento con Apache Beam

6.1. Adaptación de Kafka a micro-batches

La implementación final utiliza un consumidor persistente basado en `confluent-kafka` para obtener eventos desde Kafka.

Los eventos se reciben en lotes y cada lote se procesa mediante una ejecución finita de Apache Beam utilizando `PrismRunner`. Esta decisión fue adoptada para obtener una ejecución estable y reproducible en el entorno Windows utilizado para el proyecto.

Por tanto, la arquitectura final puede representarse como:

$$\text{Kafka} \rightarrow \text{Consumer} \rightarrow \text{Micro-batch} \rightarrow \text{Beam} \rightarrow \text{Kafka}$$

Esta implementación difiere de una arquitectura basada directamente en `KafkaIO`. La decisión permitió separar el mecanismo de ingesta de Kafka de las transformaciones realizadas por Beam, manteniendo en Beam el procesamiento por tiempo de evento, las ventanas y la agregación.

6.2. Asignación de Event Time

Cada evento es convertido a un objeto con timestamp mediante `TimestampedValue`. Conceptualmente:

$$\text{event_time} \rightarrow \text{Beam Timestamp}$$

El timestamp no se obtiene del momento en que el consumidor recibe el mensaje, sino del

campo `event_time` generado por el productor. Esto permite que un evento retrasado pueda pertenecer a una ventana anterior aunque haya sido recibido posteriormente.

6.3. Ventanas

El pipeline utiliza ventanas fijas de:

$$W = 60 \text{ segundos}$$

Por lo tanto, los eventos son agrupados en intervalos consecutivos de un minuto. La elección de ventanas de 60 segundos permite observar cambios en las métricas con una granularidad suficientemente pequeña para un escenario de monitoreo operativo y, al mismo tiempo, mantener una implementación sencilla.

6.4. Eventos tardíos

Se establece:

$$\text{AllowedLateness} = 120 \text{ segundos}$$

El productor puede generar eventos cuyo `event_time` se encuentra aproximadamente 120 segundos por detrás del tiempo actual. La finalidad es simular situaciones reales en las que un evento puede llegar después del momento en que ocurrió.

El sistema utiliza Event Time para determinar la ventana correspondiente y mantiene además un estado persistente de los eventos procesados.

7. Agregación y deduplicación

7.1. Agregación

La agregación se implementa mediante un `CombineFn` denominado `AggregatePaymentsFn`.

Para cada comercio y ventana se calculan:

$$\text{TotalEventos} = \# \text{Eventos}$$

$$\text{EventosConfirmados} = \#\{e : \text{status}(e) = \text{CONFIRMED}\}$$

$$\text{MontoTotal} = \sum_{e:\text{status}(e)=\text{CONFIRMED}} \text{amount}(e)$$

Los eventos con estado `REJECTED` o `PENDING` no se incorporan al monto total confirmado.

7.2. Deduplicación

La deduplicación se realiza utilizando `event_id` como identificador estable.

Dentro de la agregación, los eventos se almacenan temporalmente en una estructura indexada por `event_id`. Si un evento vuelve a aparecer con el mismo identificador, no se contabiliza nuevamente.

Además, debido a que el procesamiento se realiza en micro-batches, se implementó un estado persistente en SQLite. La tabla principal utiliza:

```
event_id TEXT PRIMARY KEY
```

y la inserción se realiza mediante:

```
INSERT OR IGNORE
```

Esto permite evitar que un mismo evento vuelva a incorporarse si aparece en otro micro-batch.

8. Estado persistente e idempotencia

Uno de los principales problemas identificados durante las pruebas fue que una misma ventana podía recibir eventos en diferentes micro-batches.

Por ejemplo:

Batch	Eventos nuevos	Eventos acumulados
1	5	5
2	3	8
3	2	10

Para resolver este problema se implementó un almacén de estado local basado en SQLite. El estado mantiene, entre otros datos:

- `event_id`;
- `comercio`;
- `inicio de ventana`;
- `fin de ventana`;
- `monto`;
- `estado`;
- representación JSON del evento.

De esta manera, las métricas se recalculan a partir del estado acumulado de cada comercio y ventana.

8.1. Clave de idempotencia

La clave utilizada para identificar una agregación es:

$$K = \text{merchant_id} \mid \text{window_start} \mid \text{window_end}$$

Por ejemplo:

```
merchant_alpha|Timestamp(1790705520)|Timestamp(1790705580)
```

Esta clave permite identificar de manera determinística el estado agregado correspondiente a un comercio y una ventana temporal.

9. Esquema de salida

Los resultados publicados en `aggregates.v1` tienen una estructura similar a:

```
{
  "idempotency_key": "merchant_alpha|window_start|window_end",
  "merchant_id": "merchant_alpha",
  "window_start": "Timestamp(...)",
  "window_end": "Timestamp(...)",
  "metrics": {
    "total_amount": 2415.76,
    "total_events": 19,
    "confirmed_events": 11
  }
}
```

La salida está diseñada como un registro agregado por comercio y ventana. La utilización de una clave determinística y de un tópico compactado permite materializar posteriormente el estado más reciente de una agregación.

10. Semántica de entrega

La solución no declara una garantía de *exactly-once end-to-end*. La semántica alcanzada se basa en:

- identificadores estables de eventos;
- deduplicación lógica por `event_id`;
- estado persistente en SQLite;
- clave determinística para los resultados;
- tópico Kafka de salida configurado con compactación.

Esto permite obtener un comportamiento idempotente a nivel lógico de los resultados, pero no implica que cada mensaje haya sido procesado exactamente una única vez en todos los componentes del sistema.

La distinción es importante porque Kafka, el consumidor, Beam y el almacén de estado constituyen componentes diferentes y la garantía exactly-once end-to-end requeriría demostrar coordinadamente la semántica de todos ellos.

11. Pruebas y validación

11.1. Pruebas unitarias

Se implementaron pruebas para verificar la lógica de agregación y deduplicación. Actualmente el conjunto principal de pruebas ejecutado mediante:

```
uv run pytest -q
```

presenta:

```
2 passed
```

Las pruebas verifican principalmente:

- que un evento duplicado no sea contabilizado dos veces;
- que dos eventos diferentes sean contabilizados correctamente;
- que los eventos confirmados sean incluidos en el monto total;
- que eventos rechazados no sean sumados al monto confirmado.

11.2. Validación de Event Time y ventanas

Se realizaron pruebas independientes para verificar:

- conversión de `event_time` a timestamp de Beam;
- asignación a ventanas fijas;
- procesamiento mediante PrismRunner;
- lectura de eventos desde Kafka;
- escritura de resultados en Kafka.

11.3. Escenario de duplicados

El productor incluye una probabilidad de generación de eventos duplicados. En este escenario se reutiliza el mismo `event_id`. El pipeline detecta el identificador previamente procesado y evita incorporarlo nuevamente a las métricas.

Además, SQLite proporciona una segunda capa de protección cuando el evento aparece en otro micro-batch.

11.4. Escenario de eventos tardíos

El productor puede generar eventos cuyo tiempo de evento se encuentra aproximadamente 120 segundos por detrás del tiempo actual. El pipeline utiliza el `event_time` original para determinar la ventana, en lugar de utilizar exclusivamente el momento de recepción.

De esta forma se puede observar la diferencia entre:

$$\text{Event Time} \neq \text{Processing Time}$$

11.5. Prueba end-to-end

Se realizó una ejecución real del recorrido completo:

Productor → Kafka → Consumer → Beam → SQLite → Kafka

Durante la ejecución se observaron registros de procesamiento similares a:

```
[UPSERT] merchant_alpha |
Eventos nuevos: 1 |
Eventos acumulados: 19 |
Confirmados: 11 |
Total: $2415.76
```

Estos registros permiten comprobar que el estado de una ventana puede incrementarse a medida que llegan nuevos eventos.

12. Operación y reproducibilidad

La infraestructura Kafka se ejecuta mediante Docker Compose. Los principales componentes utilizados son:

- Docker;
- Docker Compose;
- Python 3.12;
- Apache Beam;
- `confluent-kafka`;
- `kafka-python-ng`;
- `pytest`;
- `uv`.

12.1. Inicio de la infraestructura

Desde el directorio raíz del proyecto:

```
docker compose up -d
```

La ejecución puede verificarse mediante:

```
docker compose ps
```

12.2. Instalación del entorno Python

El proyecto utiliza un archivo `pyproject.toml` y un archivo `uv.lock` para facilitar la reproducción del entorno. Las pruebas pueden ejecutarse mediante:

```
uv run pytest -q
```

12.3. Ejecución del productor

Desde la raíz del proyecto puede ejecutarse el productor mediante:

```
uv run python src/producer/producer.py
```

El productor comenzará a generar eventos sobre:

```
events.v1
```

12.4. Ejecución del pipeline

En otra terminal se ejecuta:

```
uv run python src/pipeline/main.py
```

El pipeline comienza a consumir eventos, construir micro-batches y ejecutar el procesamiento de Apache Beam.

12.5. Detención

El productor y el pipeline pueden detenerse mediante:

```
Ctrl + C
```

La infraestructura Kafka puede detenerse mediante:

```
docker compose down
```

13. Logs y observabilidad

La aplicación genera registros asociados a las principales etapas del procesamiento:

- inicio y cierre del consumidor Kafka;
- recepción de eventos;

- errores de procesamiento;
- inicio y finalización de trabajos Beam;
- inicialización y cierre del estado SQLite;
- cantidad de eventos nuevos;
- cantidad de eventos acumulados;
- cantidad de eventos confirmados;
- monto total de la ventana;
- ventana temporal procesada.

Estos registros permiten observar el recorrido del evento y verificar que el sistema no se limita a una ejecución con datos ideales.

14. Supuestos

La solución adopta los siguientes supuestos:

1. Cada evento posee un `event_id` estable.
2. Los eventos duplicados conservan el mismo `event_id`.
3. `merchant_id` constituye una clave de negocio válida.
4. El campo `event_time` representa el momento real del evento.
5. Los pagos con estado `CONFIRMED` son los únicos que contribuyen al monto total.
6. La ventana de 60 segundos es adecuada para el monitoreo planteado.
7. Un retraso de hasta 120 segundos es aceptable para el escenario simulado.
8. El entorno de ejecución es local y no representa un despliegue de producción.

15. Límites conocidos

La solución presenta algunas limitaciones que deben considerarse.

15.1. Entrada a Beam

La arquitectura final utiliza un consumidor Kafka basado en `confluent-kafka` como adaptador de entrada hacia micro-batches de Apache Beam, en lugar de utilizar directamente `KafkaIO`. Esta decisión se tomó para obtener una ejecución estable y reproducible en Windows.

Por lo tanto, la solución no debe interpretarse como una implementación completamente continua basada en `KafkaIO`.

15.2. Validación formal del contrato

La implementación actual valida implícitamente determinados campos durante el procesamiento, pero no dispone de una etapa completa de validación de esquema con una rama explícita de eventos inválidos.

Una implementación futura podría incorporar una etapa:

$$\text{Eventos} \rightarrow \text{Validación} \rightarrow \begin{cases} \text{Válidos} & \rightarrow \text{Procesamiento} \\ \text{Inválidos} & \rightarrow \text{DLQ} \end{cases}$$

15.3. Persistencia local

SQLite se utiliza como mecanismo de estado local. Esta decisión resulta adecuada para el alcance académico del proyecto, pero no constituye una solución distribuida de estado para un entorno productivo.

15.4. Exactly-once

No se garantiza exactamente una única entrega de extremo a extremo. La solución proporciona deduplicación e idempotencia lógica, pero una garantía exactly-once completa requeriría una coordinación adicional entre todos los componentes.

15.5. Retención del estado

El estado SQLite no implementa actualmente una política avanzada de expiración de ventanas históricas. En un sistema de producción sería necesario definir una estrategia de retención y limpieza del estado.

16. Posibles mejoras

Como líneas de evolución del proyecto se plantean:

1. utilizar directamente `KafkaIO` dentro de Apache Beam en un entorno donde el runner y la infraestructura lo permitan de manera estable;
2. incorporar validación explícita del contrato de eventos;
3. agregar una Dead Letter Queue para eventos inválidos;
4. incorporar explícitamente un campo `schema_version`;
5. implementar pruebas formales con `TestPipeline` o `TestStream`;
6. automatizar escenarios específicos de duplicados y eventos tardíos;
7. incorporar una política de expiración del estado SQLite;
8. reemplazar SQLite por un almacén de estado distribuido en una implementación de mayor escala;

9. incorporar métricas operativas como throughput, latencia, cantidad de duplicados y cantidad de eventos tardíos;
10. implementar un consumidor de resultados que materialice la última versión de cada clave de agregación.

17. Conclusiones

El proyecto implementa un pipeline funcional de procesamiento de datos en streaming orientado al análisis de eventos de pagos. La solución integra un productor sintético, Apache Kafka, Apache Beam, procesamiento por tiempo de evento, ventanas temporales, deduplicación, persistencia de estado y publicación de resultados agregados.

Uno de los principales aspectos abordados fue la diferencia entre tiempo de evento y tiempo de procesamiento. La utilización de `event_time`, ventanas fijas de 60 segundos y un *allowed lateness* de 120 segundos permite representar escenarios en los que los eventos no llegan exactamente en el orden en que ocurrieron.

La incorporación de un estado persistente mediante SQLite permitió resolver el problema de acumulación entre micro-batches y complementar la deduplicación realizada durante la agregación. La clave determinística de salida permite identificar cada combinación de comercio y ventana, mientras que la compactación del tópico de salida favorece la materialización del estado más reciente.

La solución fue validada mediante pruebas unitarias y ejecuciones end-to-end, observándose el recorrido completo desde la generación de eventos hasta la publicación de resultados.

Finalmente, se reconoce que la implementación no constituye un despliegue de producción ni declara una garantía exactly-once end-to-end. Asimismo, la entrada a Beam se implementa mediante un consumidor Kafka persistente que alimenta micro-batches, en lugar de utilizar directamente KafkaIO. Estas decisiones y limitaciones forman parte del alcance técnico de la solución y establecen claramente las condiciones bajo las cuales deben interpretarse sus garantías.

Repositorio

El código fuente, infraestructura, pipeline y pruebas se encuentran disponibles en:

<https://github.com/diegoolmedord/proyecto-integrador-streaming>